

```

"""
constants_validator.py
=====
GRUS Audit Pipeline – Stage 2: Constants Validation
Specification: GRUS-STD-001-v1.0 Section 3 Stage 2; Clause 1

Verifies that all constants in the submission are imported from a single
source-of-truth file (typically svcf_constants.py), that no constant is
hardcoded outside that file, and that the constant values match the
expected SVCF foundation values within tolerance.

Published by: Green Recursive Utility Service LLC
"""

import os
import ast
from typing import Dict, List, Set
from dataclasses import dataclass, field

@dataclass
class ConstantsValidationResult:
    """Result of constants validation stage."""
    passed: bool
    constants_module_path: str = ""
    declared_constants: Dict[str, str] = field(default_factory=dict)
    failure_reason: str = ""
    failing_field: str = ""

# Hardcoded numeric literals that would indicate constant violations
# These are the SVCF foundation constant values; if they appear as
# numeric literals outside svcf_constants.py, that's a Clause 1 violation
SUSPICIOUS_LITERALS = {
    "1.0/2857.0",
    "32.0/33.0",
    "65.0/66.0",
    "1.01e-26",
    "6.8e-28",
    "11100",
    "37 * 300",
}

class ConstantHardcodingChecker(ast.NodeVisitor):
    """AST visitor that detects hardcoded constants outside svcf_constants.py."""

    def __init__(self, source_path: str):

```

```

self.source_path = source_path
self.violations: List[str] = []

def visit_Constant(self, node: ast.Constant) -> None:
    # Look for SVCF-foundation numeric values appearing as literals
    if isinstance(node.value, float):
        if abs(node.value - 1.01e-26) < 1e-30:
            self.violations.append(
                f"{self.source_path}:{node.lineno}: hardcoded RHO_C value 1.01e-26"
            )
        elif abs(node.value - 6.8e-28) < 1e-32:
            self.violations.append(
                f"{self.source_path}:{node.lineno}: hardcoded ETA value 6.8e-28"
            )
    self.generic_visit(node)

def find_constants_module(repo_path: str) -> str:
    """Find the constants module (svcf_constants.py or similar)."""
    candidates = ["svcf_constants.py", "constants.py", "grus_constants.py"]
    for candidate in candidates:
        full_path = os.path.join(repo_path, candidate)
        if os.path.exists(full_path):
            return full_path
    return ""

def parse_constants_module(module_path: str) -> Dict[str, str]:
    """
    Parse the constants module and extract declared constants.

    Returns a dict mapping constant name -> source-line representation.
    """
    declared: Dict[str, str] = {}
    if not module_path or not os.path.exists(module_path):
        return declared

    try:
        with open(module_path, "r", encoding="utf-8") as f:
            source = f.read()
            tree = ast.parse(source)
    except (SyntaxError, UnicodeDecodeError):
        return declared

    for node in ast.walk(tree):
        if isinstance(node, ast.Assign):
            for target in node.targets:
                if isinstance(target, ast.Name) and target.id.isupper():
                    # All-caps name = declared constant by convention
                    declared[target.id] = ast.unparse(node.value)

```

```
return declared
```

```
def check_no_hardcoding(repo_path: str, constants_module_path: str) -> List[str]:
    """
    Walk all .py files in repo, except the constants module itself,
    and check for hardcoded SVCF foundation values.
    """
    violations: List[str] = []

    for root, _, files in os.walk(repo_path):
        for fname in files:
            if not fname.endswith(".py"):
                continue
            full_path = os.path.join(root, fname)
            if os.path.abspath(full_path) == os.path.abspath(constants_module_path):
                continue
            try:
                with open(full_path, "r", encoding="utf-8") as f:
                    source = f.read()
                    tree = ast.parse(source)
            except (SyntaxError, UnicodeDecodeError):
                continue

            checker = ConstantHardcodingChecker(full_path)
            checker.visit(tree)
            violations.extend(checker.violations)

    return violations
```

```
def check_imports(repo_path: str, constants_module_name: str) -> Set[str]:
    """
    Walk all .py files and verify that any file using SVCF constants
    imports them from the constants module rather than redeclaring.
    """
    files_using_constants: Set[str] = set()
    constants_stem = os.path.splitext(constants_module_name)[0]

    for root, _, files in os.walk(repo_path):
        for fname in files:
            if not fname.endswith(".py"):
                continue
            full_path = os.path.join(root, fname)
            try:
                with open(full_path, "r", encoding="utf-8") as f:
                    source = f.read()
                    tree = ast.parse(source)
            except (SyntaxError, UnicodeDecodeError):
                continue
```

```

        for node in ast.walk(tree):
            if isinstance(node, ast.ImportFrom):
                if node.module and constants_stem in node.module:
                    files_using_constants.add(full_path)

    return files_using_constants

def validate_constants(repo_path: str, manifest: Dict) -> ConstantsValidationResult:
    """
    Full constants-validation stage.

    Args:
        repo_path: submission repository root
        manifest: parsed GRUS_MANIFEST.yaml

    Returns:
        ConstantsValidationResult
    """
    # 1. Find the constants module
    constants_module = find_constants_module(repo_path)
    if not constants_module:
        return ConstantsValidationResult(
            passed=False,
            failure_reason="No constants module found (expected svcf_constants.py or constants.py)",
            failing_field="constants_module",
        )

    # 2. Parse the constants module
    declared = parse_constants_module(constants_module)
    if not declared:
        return ConstantsValidationResult(
            passed=False,
            constants_module_path=constants_module,
            failure_reason=f"Constants module {constants_module} has no all-caps constant declarations",
            failing_field="constants_declarations",
        )

    # 3. Verify all manifest-declared constants are in the module
    manifest_constants = manifest.get("constants_used", [])
    for c in manifest_constants:
        symbol = c.get("symbol", "")
        # Allow Greek-letter aliases mapped to ASCII names
        ascii_aliases = {
            "Γ": "GAMMA", "B": "B", "β": "BETA", "Ψ": "PSI",
            "K_TD": "K_TD", "k": "K_MODE", "ρ_c": "RHO_C",
            "η": "ETA", "ε": "EPSILON",

```

```

    }
    ascii_name = ascii_aliases.get(symbol, symbol.upper())
    if ascii_name not in declared:
        return ConstantsValidationResult(
            passed=False,
            constants_module_path=constants_module,
            declared_constants=declared,
            failure_reason=f"Manifest declares constant '{symbol}' but
{os.path.basename(constants_module)} does not define {ascii_name}",
            failing_field=f"constants_used.{symbol}",
        )

# 4. Walk all .py files and check for hardcoded SVCF foundation values
violations = check_no_hardcoding(repo_path, constants_module)
if violations:
    return ConstantsValidationResult(
        passed=False,
        constants_module_path=constants_module,
        declared_constants=declared,
        failure_reason=f"Hardcoded SVCF foundation values detected outside
{os.path.basename(constants_module)}: {len(violations)} violations",
        failing_field=f"hardcoded_constants ({violations[0] if violations else ''})",
    )

# 5. Confirm files using constants actually import from the module
constants_module_name = os.path.basename(constants_module)
using_files = check_imports(repo_path, constants_module_name)
if not using_files:
    return ConstantsValidationResult(
        passed=False,
        constants_module_path=constants_module,
        declared_constants=declared,
        failure_reason=f"No files import from {constants_module_name}; constants module is
unused",
        failing_field="constants_usage",
    )

return ConstantsValidationResult(
    passed=True,
    constants_module_path=constants_module,
    declared_constants=declared,
    failure_reason="",
    failing_field="",
)

if __name__ == "__main__":
    import sys
    import yaml as _yaml
    if len(sys.argv) < 2:

```

```
    print("Usage: python constants_validator.py <repo_path>")
    sys.exit(0)
repo = sys.argv[1]
manifest_file = os.path.join(repo, "GRUS_MANIFEST.yaml")
with open(manifest_file) as f:
    manifest = _yaml.safe_load(f)
result = validate_constants(repo, manifest)
if result.passed:
    print("[PASS] Constants validation.")
    print(f"  Module: {result.constants_module_path}")
    print(f"  Declared constants: {len(result.declared_constants)}")
else:
    print(f"[FAIL] {result.failure_reason}")
    print(f"  Failing: {result.failing_field}")
    sys.exit(1)
```